

STEP 2 — Import Libraries

```
# Data Handling
import pandas as pd
import numpy as np

# Text Preprocessing
import re
import nltk
from nltk.corpus import stopwords

# Feature Extraction
from sklearn.feature_extraction.text import TfidfVectorizer

# Model Training
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Evaluation
from sklearn.metrics import accuracy_score, precision_score, recall_score, f
```

STEP 3 — Load and Preprocess Data.

Load Dataset

```
# Example: Spam SMS Dataset
df = pd.read_csv("IMDB Dataset.csv", encoding='latin-1')

df = df[['review', 'sentiment']]
df.columns = ['text', 'label']

df.head()
```

	text	label	
0	One of the other reviewers has mentioned that ...	positive	
1	A wonderful little production. The...	positive	
2	I thought this was a wonderful way to spend ti...	positive	
3	Basically there's a family where a little boy ...	negative	
4	Petter Mattei's "Love in the Time of Money" is...	positive	

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

Download Stopwords

```
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

Text Cleaning Function

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-zA-Z', ' ', text)
    words = text.split()
    words = [word for word in words if word not in stop_words]
    return " ".join(words)

df['clean_text'] = df['text'].apply(clean_text)

df[['text', 'clean_text']].head()
```

	text	clean_text
0	One of the other reviewers has mentioned that ...	one reviewers mentioned watching oz episode ho...
1	A wonderful little production. The...	wonderful little production br br filming tech...
2	I thought this was a wonderful way to spend ti...	thought wonderful way spend time hot summer we...
3	Basically there's a family where a little boy ...	basically family little boy jake thinks zombie...
4	Petter Mattei's "Love in the Time of Money" is...	petter mattei love time money visually stunnin...

STEP 4 — Feature Extraction (TF-IDF)

```
vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(df['clean_text'])
y = df['label']

print("Vocabulary Size:", len(vectorizer.vocabulary_))
```

Vocabulary Size: 99255

STEP 5 — Train Logistic Regression Model.

Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Train Model

```
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
```

▼ **LogisticRegression** ⓘ ?
 LogisticRegression(max_iter=200)

STEP 6 — Model Evaluation

```
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, pos_label='positive')
recall = recall_score(y_test, y_pred, pos_label='positive')
f1 = f1_score(y_test, y_pred, pos_label='positive')

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:\n")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.8985
Precision: 0.8878180416345413
Recall: 0.9140702520341337
F1 Score: 0.9007529089664613
```

Classification Report:

	precision	recall	f1-score	support
negative	0.91	0.88	0.90	4961
positive	0.89	0.91	0.90	5039
accuracy			0.90	10000
macro avg	0.90	0.90	0.90	10000
weighted avg	0.90	0.90	0.90	10000

Confusion Matrix:

```
[[4379 582]
 [ 433 4606]]
```

STEP 7: Analyze important words using model coefficients.

```
"""
STEP 7: Analyze important words using model coefficients

Positive coefficients → Positive sentiment words
Negative coefficients → Negative sentiment words
"""

feature_names = np.array(vectorizer.get_feature_names_out())
coefficients = model.coef_[0]

# Top 10 positive words
top_positive = feature_names[np.argsort(coefficients)[-10:]]

# Top 10 negative words
top_negative = feature_names[np.argsort(coefficients)[:10]]

print("\nTop Positive Words:")
print(top_positive)

print("\nTop Negative Words:")
print(top_negative)

Top Positive Words:
['enjoyed' 'favorite' 'loved' 'brilliant' 'amazing' 'perfect' 'wonderful'
 'best' 'excellent' 'great']

Top Negative Words:
['worst' 'bad' 'waste' 'awful' 'boring' 'poor' 'terrible' 'nothing'
 'worse' 'dull']
```

